Comparative Study of Deep Learning Models for Pork Packaging Defect Detection

Name: Daniel Law En Jie **Monash ID:** 33521875

1. Introduction and Problem Definition

1.1 Engineering Background and Motivation

Quality inspection in food processing is essential to ensure packaging standards, but conventional manual inspection may introduce inconsistencies due to subjective judgement and fatigue. Computer vision offers a potential approach to improving inspection efficiency and consistency [1]. In pork rasher packaging, defects such as **loose-meat, packaging errors, twisted-meat, and incomplete sealing** can affect product quality and integrity.

Deep learning enables automated defect identification and localisation using object detection techniques [2]. However, **it remains unclear whether larger models with more parameters provide better defect detection performance sufficient to justify their higher computational requirements** [3]. This presents an engineering trade-off between detection accuracy, processing speed, and model complexity. Therefore, **this project compares YOLOv8n, RetinaNet, and Faster R-CNN to determine whether more computationally demanding architectures improve pork rasher defect detection and which offers the most suitable accuracy–efficiency balance.**

1.2 Project Aim and Objectives

This project aims to identify a suitable deep learning approach for automated pork rasher packaging inspection. The evaluation focuses on **detection reliability, computational efficiency, and practical engineering suitability**. The specific objectives are:

- Analyse and prepare the dataset, then fine tune **three** pretrained models using consistent dataset splits.
- Evaluate mAP, precision, recall, F1 score, and per-class performance.
- Investigate detection errors and confidence-threshold sensitivity
- Compare inference speed, model complexity, and memory requirements to recommend a suitable architecture.

1.3 Project Scope and Limitations

This project is limited to supervised learning using an existing annotated dataset, rather than collecting new images or developing a complete industrial inspection system. Beyond that, all experiments are conducted offline, without physical development or production line testing. Therefore, the findings provide a comparative assessment of the selected approaches, but do not confirm reliability under real world operating conditions.

2. Dataset Analysis and Preparation

2.1 Dataset Description and Inputs/Outputs

An annotated pork rasher packaging dataset was obtained from Roboflow, containing RGB images and defect annotations divided into training, validation, and test sets. Each annotation identifies the defect class and its bounding box. **Figure 2.1.0** illustrates a packaging-error annotation, while **Table 2.1.0** summarises the computational inputs and outputs.

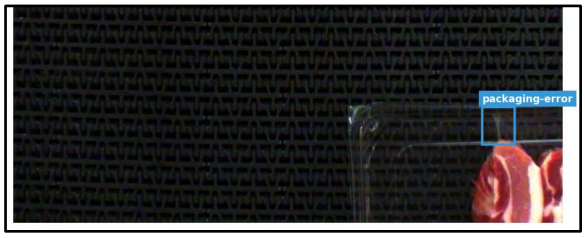


Figure 2.1.0: Example of a pork rasher packaging image with defect annotation.

Table 2.1.0: Computational inputs and outputs.

Item	Description
Image input	RGB packaging images, resized to approximately 640 × 640 pixels for model processing
Annotation input	YOLO format text files containing class IDs and normalised bounding boxes. Supported classes include loose-meat, packaging error, twisted-meat, and unsealed.
Model output	Predicted defect class, confidence score (between 0 and 1), and bounding box coordinates in image pixels.
Invalid or missing annotations	Empty labels represent images without annotated defects. Invalid boxes and unsupported classes require validation or exclusion.

The original annotations are in the **YOLO format (c, x_c, y_c, w, h)**, where c is the class ID, x and y are the coordinates, w and h are the width and height respectively. To standardize the evaluation, the coordinates are converted into **absolute pixel coordinates (x_{min}, y_{min}, x_{max}, and y_{max})**. Class labels are also converted between the zero-based YOLO convention and Torchvision indexing. After training is complete, the predicted output of the models is compared against ground truth annotations to evaluate detection and localisation performance.

2.2 Dataset Quality Audit, Preliminary Experiments, and Refinement

An initial data quality audit was conducted to evaluate dataset integrity, annotation validity, and class distribution before model training. The audit examined **image-annotation correspondence, missing or empty labels, bounding box validity, and potential overlap between augmented images,** with the results summarised in **Table 2.2.0** below**.**

Table 2.2.0: Results and findings from data audit.

Audit Item	Finding
Annotation format	YOLO-normalised bounding boxes
Empty annotations	Present, but reviewed as potential background images
Class distribution	Substantial imbalance

Significant class imbalance was identified, where **packaging error and unsealed class formed 94.58% of all annotated defects, while the remaining three made up 5.42%.** Preliminary experiments were then conducted using YOLOv8n on the original five class dataset, so that model performance can be assessed, and potential improvements can be identified. Training was done for 30 and 60 epochs, which achieved validation mAP@50:95 values of 0.2445 and 0.2903 respectively. However, map@50 decreased from 0.5321 to 0.4599, indicating that additional training did not consistently improve detection performance.

The data audit coupled with initial training **highlighted limitations in the wrinkle class**, which contained only 18 training annotations, derived from six source images, with none present in the validation or test sets. Therefore, reliable independent evaluation of this class was not possible. The data was then **refined to four classes by excluding this class,** preventing known defects from being incorrectly labelled as defect free. The final dataset used for training contained **3,338 images, comprising of 3,138 training, 120 validation, and 80 test images.** Class imbalance remained a limitation, particularly for loose-meat and twisted-meat, which only had three validation annotations each.

3. Computational Approach

3.1 Computational Workflow

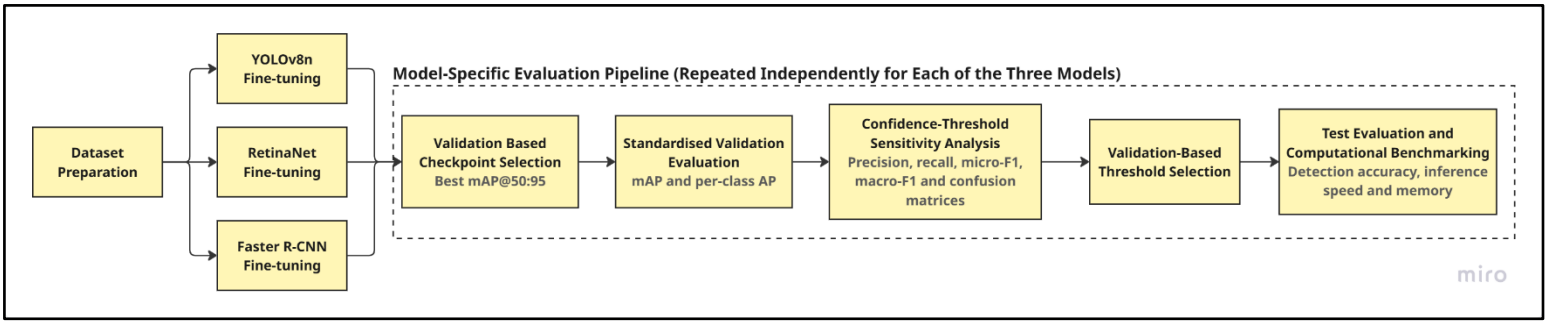


Figure 3.1.0: Block diagram of computational workflow.

The computational workflow was implemented in Python using Google Colab. Three pretrained detectors were independently fine-tuned on the same four-class dataset. Each model's best checkpoint was selected using validation mAP@50:95, followed by standardised evaluation and confidence-threshold selection using validation macro-F1. The selected configurations were then evaluated on the held-out test set and benchmarked for computational efficiency to support the final engineering assessment.

3.2 Object Detection Architectures

Three object detection architectures were selected to compare different detection mechanisms and computation requirements. Firstly, **YOLOv8n** is a lightweight, one stage detector that predicts object classes and bounding boxes without a separate region proposal stage. It uses an anchor-free detection head and multi-scale feature extraction to identify objects of different sizes [4]. The nano variant was selected as an **efficiency-oriented baseline due to its small model size and low computational requirements.**

Next, the **RetinaNet ResNet50-FPN V2** is a one-stage, anchor-based detector that uses a ResNet50 backbone and Feature Pyramid Network (FPN) to extract multi-scale image features. Its focal loss reduces the influence of easily classified background examples, which **allows training to focus on more difficult predictions** [5]. This architecture was selected to investigate **whether this approach improves defect detection under class-imbalanced conditions.**

Finally, the **Faster R-CNN ResNet50-FPN V2** is a two-stage detector that first generates candidate object regions using a Region Proposal Network (RPN), then classifies and refines their bounding boxes [6]. It was selected to **investigate whether region-based detection provides greater localisation accuracy, despite potentially higher computational costs**.

All models were fine-tuned using the same four class dataset in Google Colab. YOLOv8n was implemented using Ultralytics, while RetinaNet and Faster R-CNN used Torchvision with custom PyTorch training loops. Their main training configurations are summarised in **Table 3.2.0** below.

Table 3.2.0: Comparison of model training configurations.

Parameter	YOLOv8n	RetinaNet	Faster R-CNN
Input size	640	640	640
Batch size	16	4	4
Epochs	30	15	15
Optimiser	Auto (Ultralytics)	SGD	SGD
Initial learning rate	Automatically selected	0.005	0.005
Mixed precision	Enabled	Enabled	Enabled

YOLOv8n used Ultralytics's integrated training pipeline, whereas RetinaNet and Faster R-CNN were implemented using custom PyTorch training loops with stochastic gradient descent (SGD). The Torchvision implementations for RetinaNet and R-CNN also required converting normalised YOLO annotations into absolute pixel coordinates and shifting class indices from 0–3 to 1–4. These differences were retained and documented to accommodate their respective frameworks.

4. Validation Method and Results

4.1 Standardised Evaluation and Verification

To ensure consistent comparison, predictions from all three models were converted into a common bounding-box and class-label format before evaluation using TorchMetrics. This allowed detection performance to be assessed using the same metric implementation rather than relying on framework-specific outputs.

Beyond that, for confusion matrix analysis, predictions were matched to ground truth annotations using one-to-one Intersection over Union (IoU) matching at a threshold of 0.50. **Unmatched predictions were recorded as false positives, while unmatched ground-truth annotations were considered missed detections**. Prediction structures, class indices and bounding-box formats were also checked to minimise evaluation inconsistencies. Although a common evaluation procedure was applied, differences in architecture-specific preprocessing and non-maximum suppression remained potential limitations to direct comparison.

4.2 Evaluation Metrics

Detection performance was evaluated using Intersection over Union (IoU), precision, recall, F1 score, and mean Average Precision (mAP). These metrics assess both the ability to identify defects and the accuracy of predicted bounding boxes. The definition of each metric is stated in **Table 4.2.0** below.

Table 4.2.0: Object detection evaluation metrics.

Metric	Definition and Purpose
IoU	Ratio of overlap to union between predicted and ground truth bounding boxes, measuring localisation accuracy.
Precision	**TP/(TP + FP),** proportion of reported detections that are **correct.**
Recall	**TP/(TP + FN),** proportion of ground-truth defects **successfully detected**.
F1 score	**2PR/(P + R),** harmonic mean of precision (P) and recall (R).
mAP@50	Mean Average Precision across defect classes at IoU ≥ 0.50, evaluated across confidence rankings.
mAP@50:95	Mean Average Precision averaged over IoU thresholds from 0.50 to 0.95 in increments of 0.05, providing a more demanding assessment of localisation.

For multi-class evaluation, **micro-averaging** aggregates TP, FP, and FN across all classes before calculating the metrics, giving greater influence to frequently occurring defects, which is the packaging error and unsealed class in this case. On the other hand, **macro-averaging** calculates the metrics for each class individually before taking their unweighted mean, which gives equal importance to each defect category.

For this evaluation, both approaches were considered due to the dataset's substantial class imbalance. The micro-F1 reflects overall detection performance, while macro-F1 provides a more balanced assessment across defect class. Therefore, **macro-F1 was used to select model-specific confidence thresholds on the validation set**, reducing the influence of the dominant classes on the selection.

4.3 Validation Results and Model Selection

The best checkpoint for each architecture was selected using the highest validation mAP@50:95, rather than automatically using the final training epoch. **Figure 4.3.0** below illustrates the validation performance across training epochs, with the selected checkpoint highlighted for each model. The model weights that correspond to the strongest validation performance was also retained for evaluation.

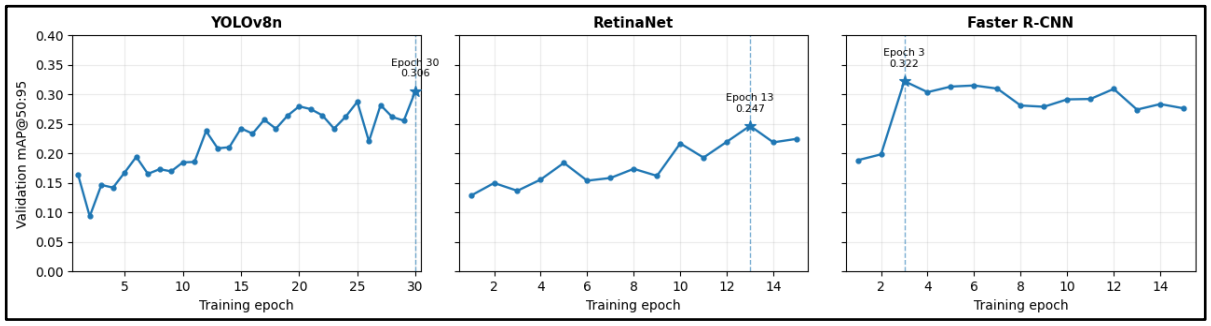


Figure 4.3.0: Validation mAP@50:95 across training epochs.

The validation mAP@50:95 values recorded during training and the corresponding selected checkpoints are summarised in **Table 4.3.0**. The selected models were subsequently evaluated using a standardised TorchMetrics implementation for direct comparison, including the per-class results presented in **Figure 4.3.1.**

Table 4.3.0: Selected checkpoints and validation performance.

Model	Training epochs	Selected epoch	Validation mAP@50:95
YOLOv8n	30	30	0.3057
RetinaNet	15	13	0.2465
Faster R-CNN	15	3	0.3222

YOLOv8n, RetinaNet, and Faster R-CNN achieved their highest training-time validation mAP@50:95 at epochs 30, 13, and 3, respectively, demonstrating the importance of validation-based checkpoint selection. **Under the standardised TorchMetrics evaluation, YOLOv8n achieved 0.3278, marginally exceeding Faster R-CNN (0.3222).**

Figure 4.3.1 below shows the standardised per-class validation AP@50:95 comparison, and demonstrates substantial class-specific differences.

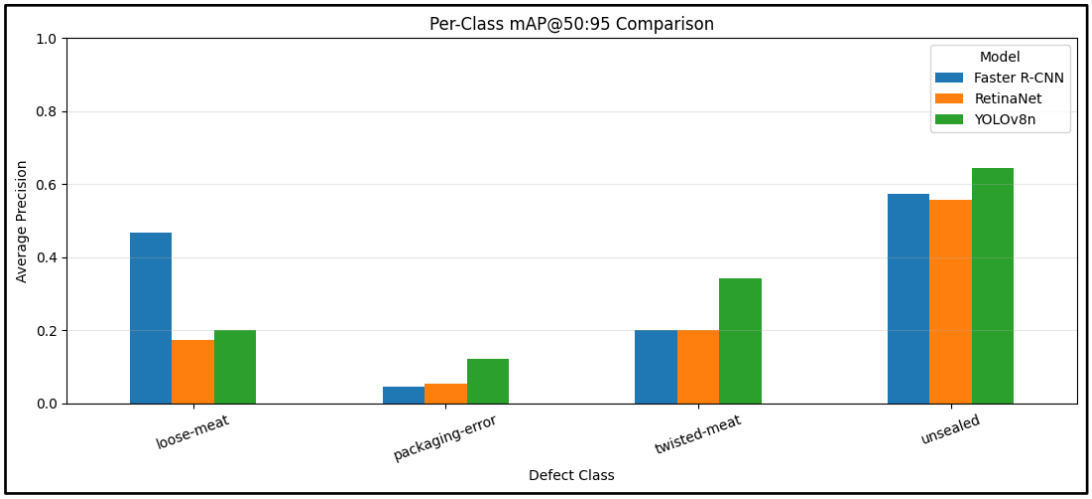


Figure 4.3.1: Standardised per-class validation AP@50:95 comparison.

Faster R-CNN performed best on loose-meat (0.4663), substantially outperforming YOLOv8n (0.2020) and RetinaNet (0.1727). However, this result was based on only three validation annotations, making the performance estimate sensitive to individual detections.

In contrast, **YOLOv8n achieved the highest AP@50:95 for packaging-error (0.1223), twisted-meat (0.3432), and unsealed (0.6437)**. Notably, packaging-error remained challenging across all three architectures, despite being the most frequently annotated class in the original dataset. This suggests that a larger number of annotations does not necessarily guarantee better detection performance, potentially due to variations in defect appearance or localisation difficulty.

All three models performed comparatively well on unsealed defects, suggesting that this class was more reliably detected within the validation dataset. Overall, these results highlight the importance of evaluating individual defect categories rather than relying solely on aggregate mAP, particularly when class distributions are highly imbalanced.

Nevertheless, validation performance alone cannot establish generalisation to unseen images, especially given the limited minority-class annotations. Final generalisation performance is therefore examined separately using the held-out test set in **Section 4.5.**

4.4 Confidence-Threshold Sensitivity

A confidence score (0-1) indicates the model's estimated certainty in a predicted detection, while the confidence threshold determines the minimum score required to retain the prediction. Confidence-threshold sensitivity was evaluated on the validation dataset across thresholds from **0.1 to 0.5 in increments of 0.025.** As shown in **Figure 4.4.0** below, increasing the threshold generally improved precision as it removed uncertain detections, while recall decreases as more defects were excluded.

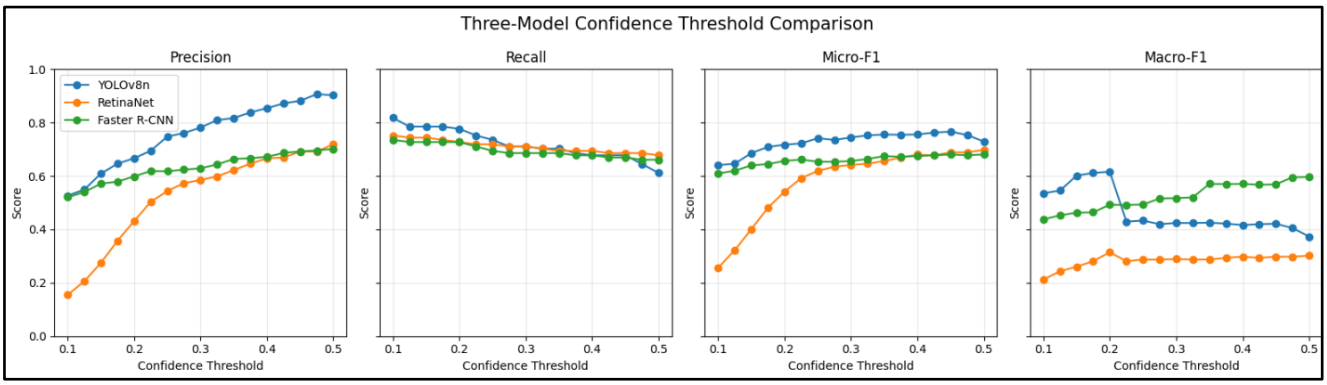


Figure 4.4.0: Confidence-threshold sensitivity of the three models.

The micro-F1 and macro-F1 trends differed due to dataset class imbalance. For YOLOv8n, micro-F1 remained relatively high at increased thresholds, whereas macro-F1 peaked at 0.20 before declining substantially, indicating reduced performance across certain defect classes. In contrast, Faster R-CNN achieved its highest macro-F1 near 0.50, suggesting that removing lower-confidence predictions reduced false positives without substantially affecting recall across the evaluated classes. This highlights differences in how each model assigns confidence scores to its predictions. **Table 4.4.0** below shows the best recorded micro and macro F1 values and the corresponding threshold.

Table 4.4.0: Validation-based confidence-threshold selection.

Model	Maximum micro-F1	Maximum macro-F1	Selected threshold
YOLOv8n	0.7664	0.6161	0.20
RetinaNet	0.6979	0.3142	0.20
Faster R-CNN	0.6809	0.5962	0.50

The **operating thresholds were selected by maximising macro-F1, giving equal importance to each defect category** rather than allowing frequently occurring classes to dominate threshold selection. These thresholds were fixed before final evaluation on the held-out test set.

4.5 Held-Out Test Results

The three selected models were evaluated on the held-out test dataset containing **80 images and 79 annotated defects**. The model checkpoints and confidence thresholds were fixed based on validation performance before final testing to avoid test-based optimisation. The test results revealed differences in mAP values and detection reliability at the selected confidence thresholds, as seen in **Table 4.5.0**.

Table 4.5.0: Held-Out Test Detection Performance.

Model	mAP@50	mAP@50:95	Precision	Recall	Micro-F1	Macro-F1
YOLOv8n	0.5252	0.2698	0.6923	0.6835	0.6879	0.3321
RetinaNet	0.2797	0.1494	0.4262	0.6582	0.5174	0.2704
Faster R-CNN	0.2716	0.1482	0.7273	0.6076	0.6621	0.2890

The table shows that all three models experienced a decline in mAP@50:95 between validation and testing. **YOLOv8n achieved the highest test mAP@50:95 of 0.2698**, decreasing from 0.3278 during validation. RetinaNet decreased from 0.2465 to 0.1494, while **Faster R-CNN experienced the largest decline, from 0.3222 to 0.1482**. This corresponds to a **relative reduction of approximately 54%**, suggesting weaker generalisation to unseen images despite its competitive validation performance.

YOLOv8n achieved the highest test recall and micro-F1, indicating the strongest overall balance between correctly identifying defects and limiting false detections. **Faster R-CNN recorded the highest precision (0.7273), but its lower recall (0.6076)** indicates that it missed more annotated defects. RetinaNet maintained relatively high recall (0.6582), although its lower precision (0.4262) indicates a greater proportion of incorrect detections.

Per-class test AP@50:95, presented in **Figure 4.5.0** below, further highlighted differences in generalisation. **YOLOv8n achieved the highest AP across all four defect categories**, including **loose-meat (0.2608), packaging-error (0.1481), twisted-meat (0.0921), and unsealed (0.5782).** In contrast, RetinaNet and Faster R-CNN achieved near zero AP for loose-meat and twisted-meat, despite maintaining comparatively strong performance on unsealed defects.

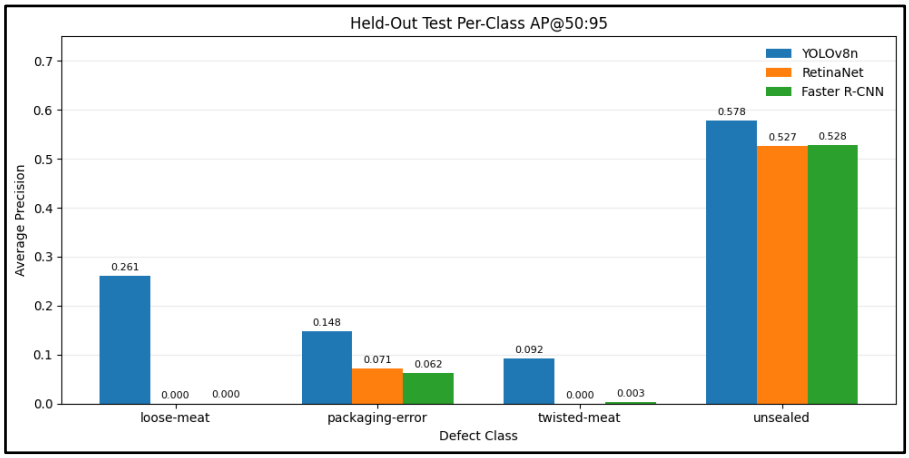


Figure 4.5.0: Per-class AP@50:95 comparison using the held-out test dataset.

These results indicate that overall detection performance can conceal substantial weaknesses in minority class recognition. However, the small number of minority class test annotations limits confidence in the exact performance differences. The detection errors and their engineering implications are examined further in **Sections 5 and 6.**

5. Comparison and Optimisation

5.1 Comparative Detection Performance

Table 5.1.0 summarises the detection errors across the three architectures at their validation-selected confidence thresholds, while **Figure 5.1.0** presents the column-normalised confusion matrices.

Table 5.1.0: Detection errors on the held-out test dataset.

Model	True Positives (TP)	False Positives (FP)	False Negatives (FN)
YOLOv8n	54	24	25
RetinaNet	52	70	27
Faster R-CNN	48	18	31

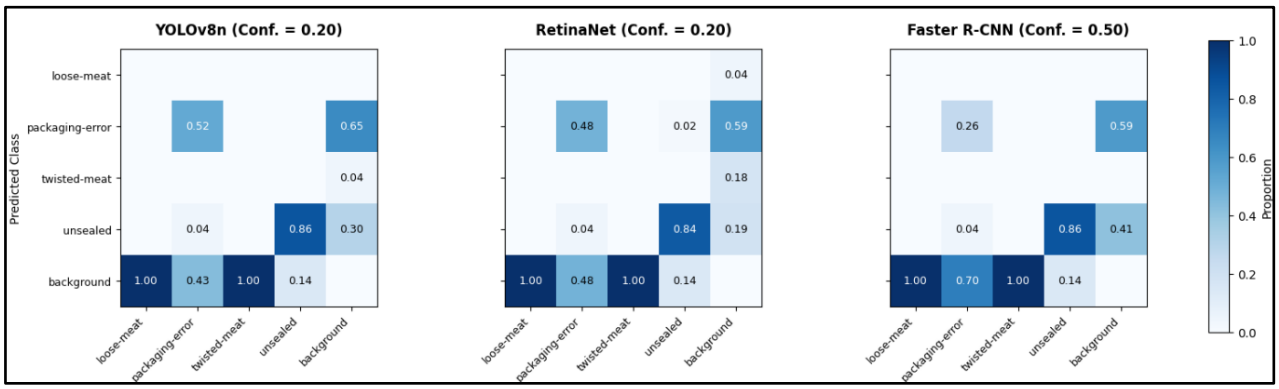


Figure 5.1.0: Column-normalised test confusion matrices for three object detection models.

YOLOv8n demonstrated the strongest overall detection balance, recording the greatest number of true positives (54) and fewest false negatives (25). RetinaNet detected a comparable number of defects (52) but generated considerably more false positives (70), potentially increasing unnecessary manual inspection or rejection. Faster R-CNN produced the fewest false positives (18) but missed the most defects (31), reflecting a more conservative detection behaviour.

Although YOLOv8n achieved nonzero test AP@50:95 for loose-meat (0.2608) and twisted-meat (0.0921), the confusion matrices revealed that **all three models failed to correctly identify loose-meat and twisted-meat defects at their selected confidence thresholds**, with all three loose-meat and four twisted-meat test annotations recorded as missed detections. This difference arises because AP evaluates predictions across confidence rankings and multiple IoU thresholds, whereas the confusion matrix considers only predictions retained at a fixed confidence threshold. Consequently, nonzero AP does not guarantee successful detection at the selected operating point.

For **packaging-error**, YOLOv8n correctly detected 12 of 23 annotations (52.2%), compared with 11 (47.8%) for RetinaNet and six (26.1%) for Faster R-CNN. In contrast, **unsealed defects were detected more consistently**, with YOLOv8n and Faster R-CNN each identifying 42 of 49 annotations (85.7%), compared with 41 (83.7%) for RetinaNet.

Overall, these findings highlight the **trade-off between false positives and missed defects**. In automated packaging inspection, **false negatives may allow defective products to pass undetected, while false positives increase unnecessary review or rejection**. Although YOLOv8n achieved the strongest overall detection balance, its failure to detect minority defects at the selected operating threshold indicates that further improvements are required before industrial deployment.

5.2 Computational Efficiency

Computational efficiency was evaluated to assess the suitability of each architecture for automated packaging inspection, where processing speed and hardware requirements are important considerations. **Table 5.2.0** below compares model complexity, inference latency, processing throughput, and recorded GPU memory usage.

Table 5.2.0: Computational Efficiency Comparison.

Model	Parameters (million)	Model weight size (MB)	Inference time (ms/image)	Processing throughput (FPS)	Recorded peak GPU memory (MB)
YOLOv8n	3.14	12.01	11.94	83.75	377
RetinaNet	36.41	139.11	59.78	16.73	425
Faster R-CNN	43.27	165.29	104.06	9.61	826

YOLOv8n demonstrated the highest computational efficiency, processing approximately 83.75 frames per second (FPS), compared with 16.73 FPS for RetinaNet and 9.61 FPS for Faster R-CNN. This corresponds to approximately **5.0x and 8.7x higher throughput** respectively. Furthermore, YOLOv8n contained only 3.14 million parameters, substantially fewer than RetinaNet (36.41 million) and Faster R-CNN (43.27 million), resulting in a smaller model weight file.

As illustrated in **Figure 5.2.0** below, **YOLOv8n achieved both the lowest inference latency (11.94 ms/image) and highest held-out test mAP@50:95 (0.2698).** In contrast, the larger architectures required greater computational resources without achieving superior detection performance. This suggests that increased model complexity does not necessarily improve detection accuracy for the investigated application.

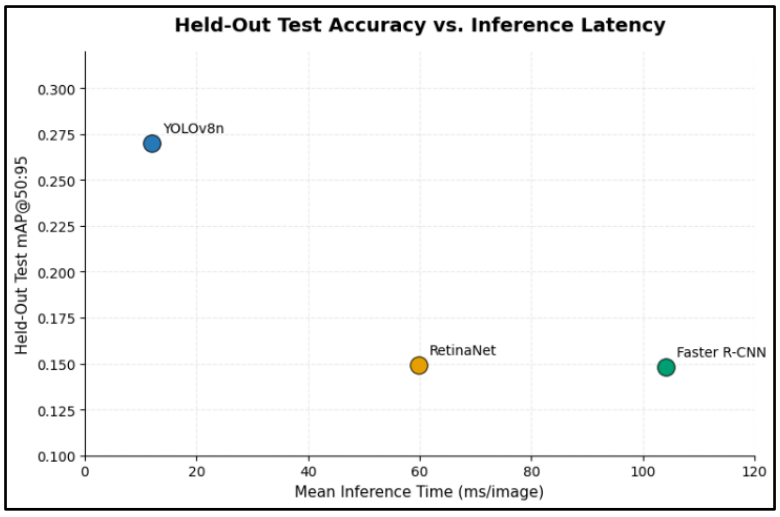


Figure 5.2.0: Relationship between held-out test mAP@50:95 and inference latency.

The recorded GPU memory values were obtained in a shared runtime and may not represent the standalone memory requirements of individual models. Additionally, differences in preprocessing and inference pipelines limit direct hardware-level comparisons. Therefore, the measured FPS values indicate relative efficiency under the experimental conditions rather than confirmed production-line performance.

5.3 Engineering Selection of the Preferred Model

The experimental workflow **incorporated optimisations such as dataset refinement, transfer learning, mixed-precision training where supported, and validation-based checkpoint and**

confidence-threshold selection. These techniques supported computational efficiency and consistent model selection without using the held-out test set for optimisation.

YOLOv8n provided the strongest observed accuracy–efficiency balance, achieving the highest test mAP@50:95 (0.2698), lowest inference latency (11.94 ms/image), and smallest model size (12.01 MB). However, its limited minority-class detection reliability and the absence of production-line testing prevent claims of industrial readiness. Therefore, YOLOv8n was selected as the preferred architecture for further development, subject to additional data collection and deployment validation.

6. Limitations, Lessons Learned and Conclusion

The main limitation was **dataset imbalance**, with packaging-error and unsealed dominating the annotations, while loose-meat and twisted-meat had very limited independent examples. Their small validation and test sets reduced the reliability of class-specific performance estimates. Furthermore, annotation inconsistencies and the use of augmented images may affect data quality and independence. As evaluation was conducted offline on a limited dataset, the results cannot be assumed to generalise to different packaging conditions or industrial environments.

Further investigation of Faster R-CNN revealed that loose-meat and twisted-meat predictions generally had very low confidence scores. However, the IoU diagnostics also showed imperfect localisation, indicating that reducing confidence thresholds alone would not guarantee reliable detection. Although class imbalance and limited visual diversity may have contributed to these failures, their exact causes were not experimentally established. This highlighted the importance of examining prediction confidence and localisation quality rather than relying solely on aggregate mAP.

Future work should prioritise **collecting more independent examples of minority defects, improving annotation consistency, and evaluating performance under different lighting, camera positions and packaging conditions**. Production-line testing should also establish acceptable false-negative rates and end-to-end processing requirements.

Overall, YOLOv8n provided the strongest observed accuracy-efficiency balance, achieving a **held-out test mAP@50:95 of 0.2698 and an inference latency of 11.94 ms/image**. However, its limited minority-class detection performance indicates that further development and validation are required before industrial deployment.

7. AI Use and Critical Reflection

ChatGPT was used as a supporting tool throughout the project, primarily to assist with code development, debugging, technical explanations and grammatical refinement of the report. It also suggested candidate object detection architectures for comparison. AI assistance was moderate, particularly for coding and writing support, to improve development efficiency and clarify unfamiliar concepts. However, **the overall workflow was independently researched and developed, while dataset refinement, experimental design, model evaluation and engineering decisions remained my responsibility**.

AI-generated code and explanations were checked through execution, debugging, comparison with actual results and independent review. One limitation was that AI occasionally suggested unnecessarily expanding the project scope or exploring approaches beyond the intended objectives, requiring critical judgement to maintain focus. Overall, AI improved development efficiency and supported my understanding of model evaluation, while reinforcing the importance of independently validating generated suggestions.

References

[1] T. Brosnan and D.-W. Sun, “Improving quality inspection of food products by computer vision—A review,” Journal of Food Engineering, vol. 61, no. 1, pp. 3–16, 2004. doi: 10.1016/S0260-8774(03)00183-3.

[2] L. Zhu, P. Spachos, E. Pensini, and K. N. Plataniotis, “Deep learning and machine vision for food processing: A survey,” Current Research in Food Science, vol. 4, pp. 233–249, 2021. doi: 10.1016/j.crfs.2021.03.009.

[3] “Deep learning in food quality and safety management: A review of architectures, applications, and future directions,” Current Research in Food Science, vol. 13, article 101495, 2026. doi: 10.1016/j.crfs.2026.101495.

[4] Ultralytics, “YOLO Architecture Explained,” Ultralytics Documentation. [Online]. Available: https://docs.ultralytics.com/guides/yolo-architecture/

[5] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, “Focal Loss for Dense Object Detection,” Proceedings of the IEEE International Conference on Computer Vision (ICCV), pp. 2980–2988, 2017. [Online]. Available: https://arxiv.org/abs/1708.02002.

[6] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks,” IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 39, no. 6, pp. 1137–1149, 2017. doi: 10.1109/TPAMI.2016.2577031.